

Course 5: Foundation Models



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Last session

- 1 Deep Learning basics
- 2 Multi-Layer Perceptron
- 3 Convolutional Neural Networks
- 4 Transformers (announced)
- 5 Deep Learning today

Today's session

- What is a Foundation Model?
- Transformers
- Self-Supervised Learning
- Multimodal Foundation Models
- Adapting a Foundation Model
- (Bonus) From LLMs to Assistants and Agents

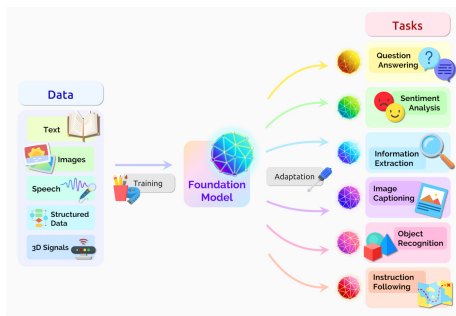
- 1 What is a Foundation Model?
- 2 Transformers
- 3 Self-supervised Learning
- 4 Multimodal Foundation Models
- 5 Adapting a Foundation Model
- 6 (Bonus) From LLMs to Assistants and Agents
- 7 Conclusion

What is a Foundation Model?

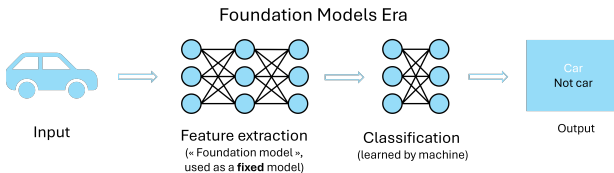
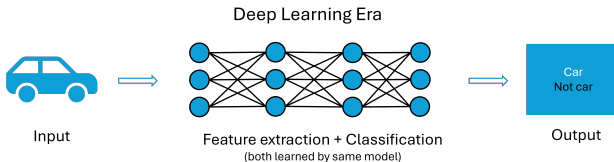
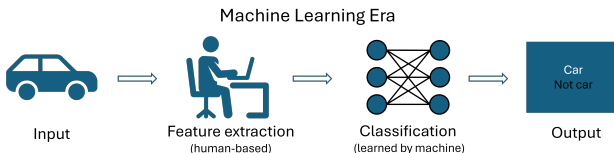
Tentative definition

- A **single** model,
- Pre-trained **once** on **internet-scale** data,
- That serves as a **generic feature representation** for many tasks.

Generalization is (mostly) solved. The problem becomes *particularization*.



What is a Foundation Model?



Where do Foundation Models come from?

Ingredient 1: Transformer

The Transformer architecture (2017). Uses attention rather than convolution, and one single architecture for text, images, audio, ...

Ingredient 2: Self-supervised learning

The labels are produced by the data itself, so the dataset is limited only by what you can download.

Ingredient 3: Scale

Data, parameters, and compute, at a massive level. "Scaling laws" tell you that the more you scale, the better the model gets.

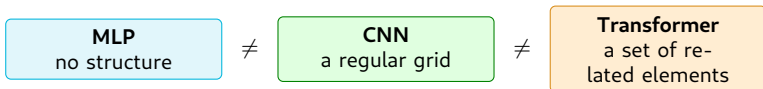
- 1 What is a Foundation Model?
- 2 Transformers
- 3 Self-supervised Learning
- 4 Multimodal Foundation Models
- 5 Adapting a Foundation Model
- 6 (Bonus) From LLMs to Assistants and Agents
- 7 Conclusion

Where the Transformer stands

The standard architecture today

- **No convolution**, no fixed neighbourhood,
- Based on *attention*: **which** elements matter for **which**?
- Used for text, images, audio, video, protein sequences, ...

what does the architecture assume about the data? (session 4)



The intuition behind attention

Attention asks one question, for each element

"Which other elements do I need to look at, to understand this one?"

The animal didn't cross the street because **it** was too tired.

A diagram illustrating attention weights for the word "it". A thick blue curved arrow originates from the word "it" and points to the word "animal". A thinner grey curved arrow originates from the word "it" and points to the word "street".

"it" → the **animal** (an animal can be tired).

One word changed at the end, and the attention weights changed.

Adapted from:

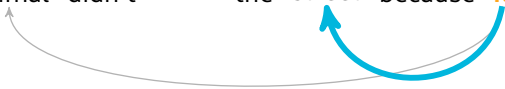
<https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/>

The intuition behind attention

Attention asks one question, for each element

"Which other elements do I need to look at, to understand this one?"

The animal didn't cross the street because **it** was too wide.



"it" → the **street** (a street can be wide).

One word changed at the end, and the attention weights changed.

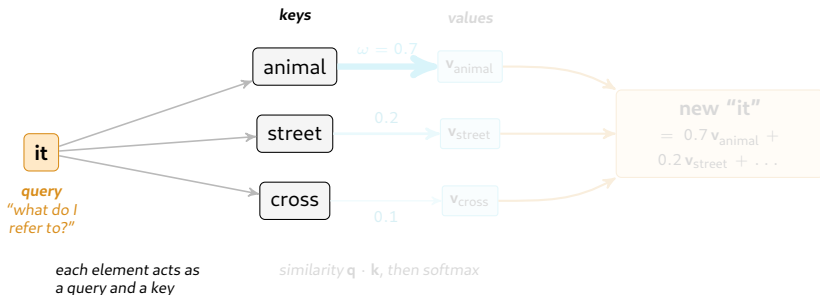
Adapted from:

<https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/>

Attention in practice

Keys, Queries, Values

- **Key** and **Query**: their similarity gives **context**.
- **Value**: integrates the context into the representation of the element.

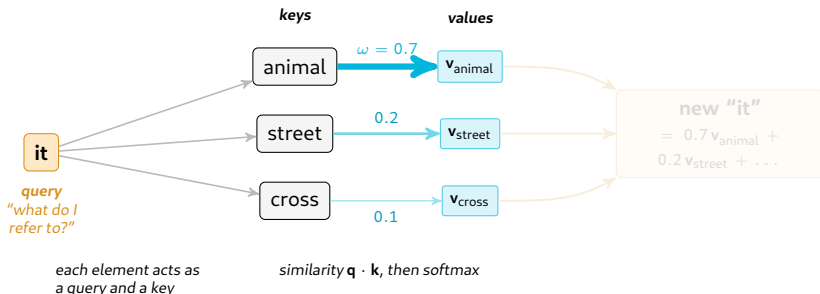


Output = an average of the values, weighted by the matches.

Attention in practice

Keys, Queries, Values

- **Key** and **Query**: their similarity gives **context**.
- **Value**: integrates the context into the representation of the element.

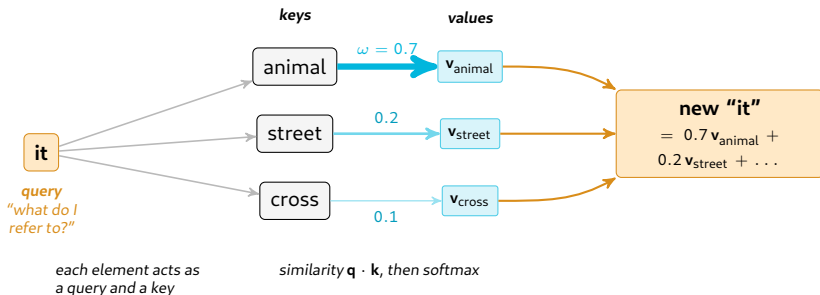


Output = an average of the values, weighted by the matches.

Attention in practice

Keys, Queries, Values

- **Key** and **Query**: their similarity gives **context**.
- **Value**: integrates the context into the representation of the element.



Output = an average of the values, weighted by the matches.

Self-attention, formally (1/2): one element at a time

From the drawing to the maths

- Each input $\mathbf{x}_i$ is projected by three **learned** matrices:

$$\mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i, \quad \mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i, \quad \mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i$$

- Attention weights $\omega_{i,j}$: the dot product (i.e. a similarity) of query i with key j , normalized by a softmax

$$\omega_{i,j} = \text{softmax}_j \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} \right)$$

- Output for element i :

$$\mathbf{z}_i = \sum_j \omega_{i,j} \mathbf{v}_j$$

Self-attention, formally (2/2): the whole input at once

Stack the elements into matrices

- Input $\mathbf{X}$ holds all elements, and $\mathbf{Q} = \mathbf{X}\mathbf{W}_q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_k$, $\mathbf{V} = \mathbf{X}\mathbf{W}_v$,
- Output $\mathbf{Z}$ is an **attention-weighted version of the input**, w.r.t. all other elements of the input.

$$\mathbf{Z} = \underbrace{\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)}_{\text{all the } \omega_{i,j} \text{ at once}} \mathbf{V}$$

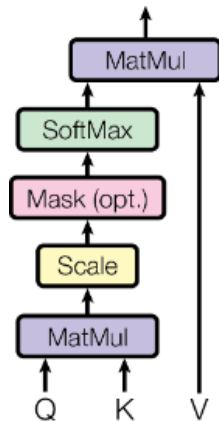


Image source: Vaswani, A. et al. (2017). Attention is all you need. Advances in neural information processing systems, 30.

Repeat, and repeat: the Transformer block

Several heads in parallel: **multi-head** attention

One attention = **one** kind of relation.

Several heads: several concepts (who does what, what refers to what, which words agree, ...).

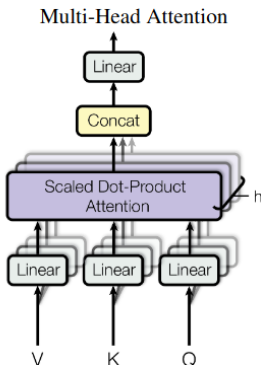


Image source: Vaswani, A. et al. (2017). Attention is all you need. Advances in neural information processing systems, 30.

Repeat, and repeat: the Transformer block

Stack the blocks: a **deep** model

Attention + a small MLP = one **block**. Repeat it N times (12, 32, ...)

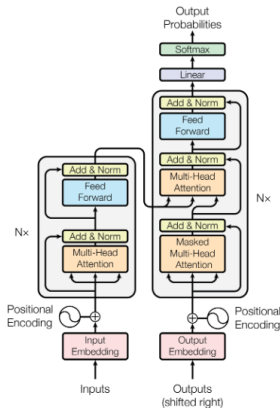


Figure 1: The Transformer - model architecture.

Image source: Vaswani, A. et al. (2017). Attention is all you need. Advances in neural information processing systems, 30.

- 1 What is a Foundation Model?
- 2 Transformers
- 3 Self-supervised Learning**
- 4 Multimodal Foundation Models
- 5 Adapting a Foundation Model
- 6 (Bonus) From LLMs to Assistants and Agents
- 7 Conclusion

Self-Supervised Learning

The intuition

Corrupt a part of the data, and train the network to reconstruct it.

⇒ the label is **produced by the data itself**: No annotator.

Pretext tasks

The artificial task built this way is called a **pretext task**.

⇒ Nobody cares about solving it, the interesting part is the **representation** learned to solve it!

Why it changes everything

- **Scale**: the dataset is limited only by what you can download, not by an annotation budget!
- **Transferability**: features learned this way are **generic**, and often transfer better than features learned on one label space.

Self-Supervised Learning

The intuition

Corrupt a part of the data, and train the network to reconstruct it.

⇒ the label is **produced by the data itself**: No annotator.

Pretext tasks

The artificial task built this way is called a **pretext task**.

⇒ Nobody cares about solving it, the interesting part is the **representation** learned to solve it!

Why it changes everything

- **Scale**: the dataset is limited only by what you can download, not by an annotation budget!
- **Transferability**: features learned this way are **generic**, and often transfer better than features learned on one label space.

Examples of pretext tasks

1. Predict the hidden part
mask a piece, guess it

BERT, GPT, MAE

2. Compare examples
same thing close,
different things apart

SimCLR, CLIP

3. Match a teacher
two views, one must
predict the other

BYOL, DINOv2/v3

4. Denoise
rebuild a corrupted input

autoencoders, diffusion

(a) Mask in the middle How are ? doing today → predict **"you"** (BERT, 2018: *understanding*)

(b) Mask what comes next How are you doing ? → predict **"today"** (GPT and all LLMs: *generation*)

BERT: <https://arxiv.org/pdf/1810.04805>

Examples of pretext tasks

1. Predict the hidden part
mask a piece, guess it

BERT, GPT, MAE

2. Compare examples
same thing close,
different things apart

SimCLR, CLIP

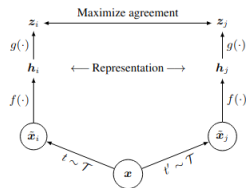
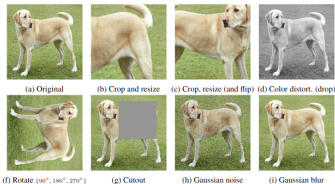
3. Match a teacher
two views, one must
predict the other

BYOL, DINOv2/v3

4. Denoise
rebuild a corrupted input

autoencoders, diffusion

- Pull together **same** data (*positives*),
- Push apart **different** data (*negatives*).



SimCLR: <https://arxiv.org/pdf/2002.05709>

Examples of pretext tasks

1. Predict the hidden part

mask a piece, guess it

BERT, GPT, MAE

2. Compare examples

same thing close,
different things apart

SimCLR, CLIP

3. Match a teacher

two views, one must
predict the other

BYOL, DINOv2/v3

4. Denoise

rebuild a corrupted input

autoencoders, diffusion

in all four cases: the supervision comes from the data itself

Last ingredient: **scale**

Orders of magnitude

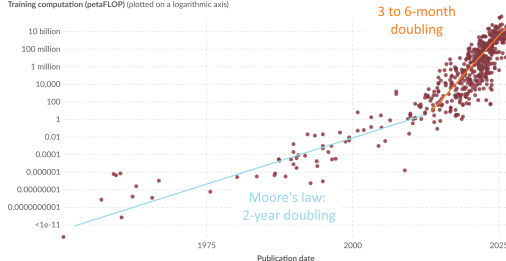
- AlexNet (2012): 62 **million** parameters,
- GPT-3 (2020): 175 **billion**,
- DeepSeek V4 Pro (2026): ≈ 1.6 **trillions** stored, but $\approx 50\text{B}$ when used.

Computation used to train notable artificial intelligence systems

Computation is measured in total petaFLOP, which is 10^{15} floating-point operations¹. Estimated from AI literature, albeit with some uncertainty. Estimates are expected to be accurate within a factor of 2, or a factor of 5 for recent undisclosed models like GPT-4.

Our World
in Data

Training computation (petaFLOP) (plotted on a logarithmic axis)



Data source: Epoch AI (2026)

OurWorldinData.org/artificial-intelligence | CC BY

1. Floating-point operation A floating-point operation (FLOP) is a type of computer operation. One FLOP represents a single arithmetic operation involving floating-point numbers, such as addition, subtraction, multiplication, or division.

- 1 What is a Foundation Model?
- 2 Transformers
- 3 Self-supervised Learning
- 4 Multimodal Foundation Models**
- 5 Adapting a Foundation Model
- 6 (Bonus) From LLMs to Assistants and Agents
- 7 Conclusion

What is a **Multimodal** Foundation Model?

The same definition, one word added

- A **single** model,
- Pre-trained **once** on **internet-scale multimodal** data.
 - Pairs (image, caption)
 - Pairs (audio, description)
 - ...
- That serves as a **generic feature representation** for many tasks.

Result: **one shared representation space** for the two modalities.

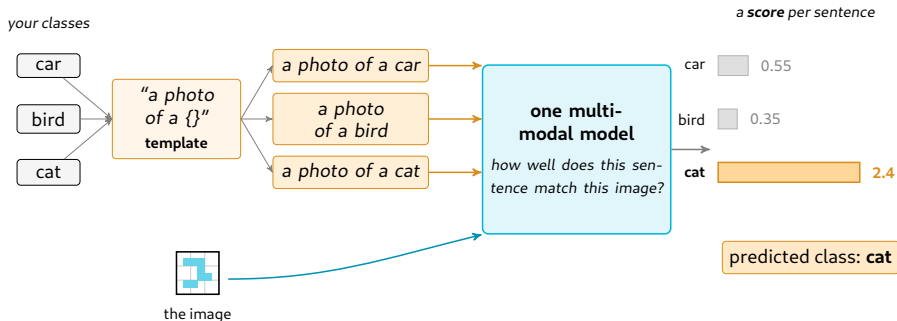
Why it matters

A concept learned in one modality becomes usable **in the other!**

Usage of text + image models

Zero-shot classification

Write the classes as **sentences**, and let the model score each one against the image. Take the best. **No training, no labelled example.**



Usage of text + image models

New tasks, beyond classification and segmentation

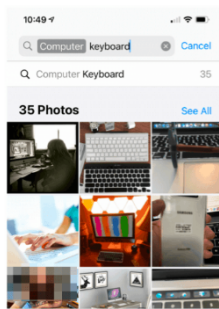
Multimodal models allow to go beyond standard tasks, and to do things that were out of reach before!

Text to image generation

a teddy bear on a skateboard in times square



Image retrieval

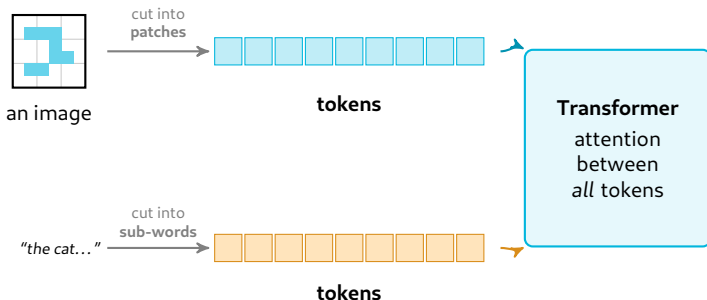


How to get multimodality (1/2)? One Transformer

Transformers are modality-agnostic

Transformers can work both on images and text!

So multimodality can be obtained by processing both at the same time!



Transformers are also data hungry

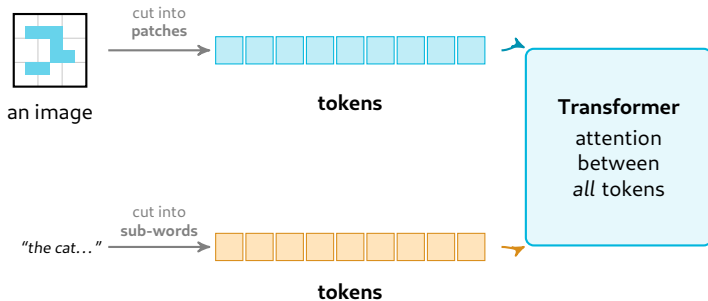
Powerful approach, but requires **massive amounts** of pairs to train it.

How to get multimodality (1/2)? One Transformer

Transformers are modality-agnostic

Transformers can work both on images and text!

So multimodality can be obtained by processing both at the same time!



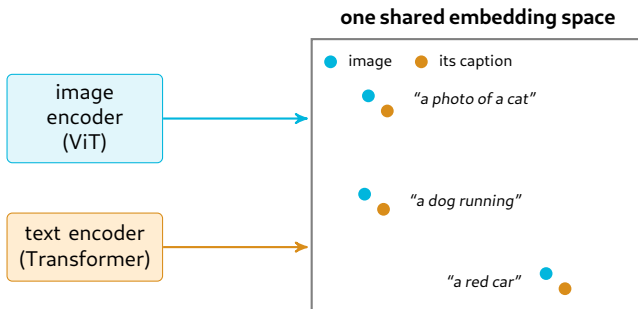
Transformers are also data hungry

Powerful approach, but requires **massive amounts** of pairs to train it.

How to get multimodality (1/2)? Two encoders

Using already trained encoders

- 1 Train encoder on each modality separately (image, text, audio, ...),
- 2 Project the outputs into a **shared embedding space**,
- 3 Train so that pairs are **close** in that space.



Multimodal models: a quick tour of what's possible

 Text  Image  Audio  Video

Between two modalities

-  $\rightarrow$  **DALL-E, Midjourney** *(generate)*
-  $\rightarrow$  **LLaVA, GPT-4 with vision** *(describe, answer)*
-  $\leftrightarrow$  **CLIP** *(search, zero-shot)*
-  $\rightarrow$  **ElevenLabs, Suno** *(speech, music)*
-  $\rightarrow$  **Whisper** *(transcribe)*
-  $\leftrightarrow$  **CLAP** *(search, zero-shot)*
-  $\rightarrow$  **Sora, Veo** *(generate)*

Going further

-     $\rightarrow$    **GPT-4o, Gemini, Qwen-Omni** *(any-to-any, one model)*

Not exhaustive, and it will be outdated in months: what matters is having in mind all the possibilities!

- 1 What is a Foundation Model?
- 2 Transformers
- 3 Self-supervised Learning
- 4 Multimodal Foundation Models
- 5 Adapting a Foundation Model**
- 6 (Bonus) From LLMs to Assistants and Agents
- 7 Conclusion

Adapting a model?

The situation

Foundation models extract **general-purpose** representations and understand your data at a high level.

→ But it is not tuned for **your** set-up! (e.g., medical imaging, which requires a precise understanding of MRI and scanners.)

Solution: Adaptation

“Adapting” a model means tailoring it to one’s specific needs.
 (“Particularization”)

Why bother, rather than training your own?

It saves an enormous amount of data, computation and time. You are **transferring** the knowledge from the general task to your specific one.

Adapting a model?

The situation

Foundation models extract **general-purpose** representations and understand your data at a high level.

→ But it is not tuned for **your** set-up! (e.g., medical imaging, which requires a precise understanding of MRI and scanners.)

Solution: Adaptation

“Adapting” a model means tailoring it to one’s specific needs.
 (“Particularization”)

Why bother, rather than training your own?

It saves an enormous amount of data, computation and time. You are **transferring** the knowledge from the general task to your specific one.

Adapting a model?

The situation

Foundation models extract **general-purpose** representations and understand your data at a high level.

→ But it is not tuned for **your** set-up! (e.g., medical imaging, which requires a precise understanding of MRI and scanners.)

Solution: Adaptation

"Adapting" a model means tailoring it to one's specific needs.
("Particularization")

Why bother, rather than training your own?

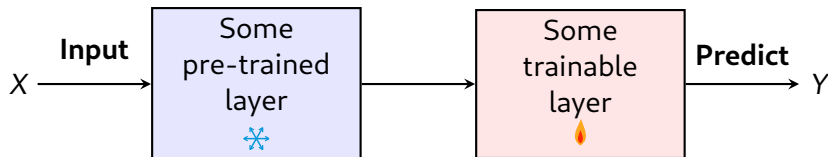
It saves an enormous amount of data, computation and time. You are **transferring** the knowledge from the general task to your specific one.

Trainable and non-trainable parameters

In adaptation, the model parameters (weights) may be trained or kept fixed ("frozen", *i.e.* they keep the values of the pre-trained model).

- 🔥 Trained parameters are denoted with a 🔥 symbol;
- ❄️ Frozen (fixed) parameters are denoted with a ❄️ symbol.

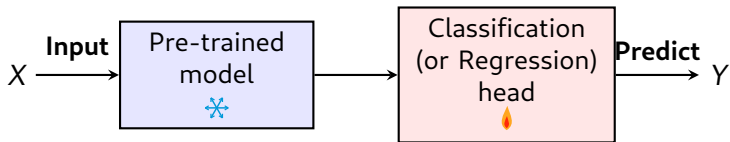
Example of a model representation:



Transfer Learning (linear probing)

Tentative definition

- Pre-trained model: **fixed feature extractor**,
- Add one (or several) layers on top of the representation.



Advantages

- Easy, model used off-the-shelf,
- Training cost depends on the head only,
- Works with **small** datasets,
- Only needs **inference** $\Rightarrow$ works with a closed model.

Disadvantages

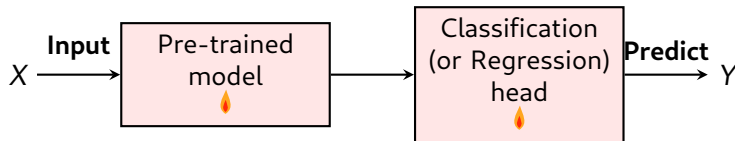
- Heavily depends on the pre-training,
- Representations may not be expressive enough for your task,
- May not benefit from a **large** dataset.

This is exactly what you do in lab 5 with frozen ViT embeddings.

Fine-tuning

Tentative definition

Re-train the **whole** model (with or without additional layers) on your data.



Advantages

- Very effective,
- Generally the **best** performance,
- Benefits from large datasets.

Disadvantages

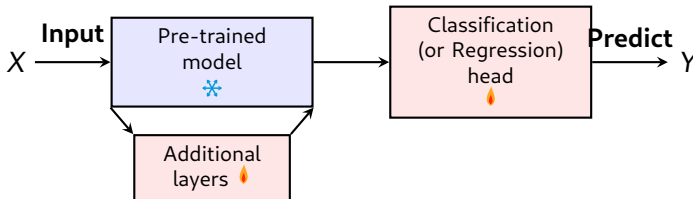
- Computationally expensive (sometimes prohibitive),
- Can destroy the pre-training (**catastrophic forgetting**),
- Requires a large dataset,
- Requires access to the weights,

Rule: it must beat the frozen probe. Otherwise it failed.

Parameter-Efficient Fine-Tuning (PEFT)

Tentative definition

Train only a small set of **added parameters**.



Advantages

- Nearly the quality of fine-tuning,
- **Lightweight**: often less than 1% of the parameters,
- One small file per task, one shared backbone,
- Scales with small **and** large datasets.

Disadvantages

- Needs some parametrization (where to insert, which rank, ...),
- Requires access to the weights,
- Adapts the model only **lightly**.

The usual best trade-off in practice.

Focus on LoRA (PEFT)

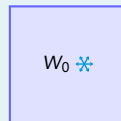
Low-Rank Adaptation

- Freeze the pre-trained matrix W_0
- Learn a **small correction**, written as $A \times B$ ($A \in \mathbb{R}^{h \times r}$ and $B \in \mathbb{R}^{r \times h}$, with r chosen by you ($r = 4, 8, 16, \dots$)).

Pre-trained weights

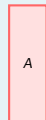
LoRA weights

$$x^{(l+1)} = W_0 x^{(l)} + AB x^{(l)}$$



$h \times h$ (large)

+



B

$h \times r$ and $r \times h$
(thin: $r \ll h$)

Example: a ViT

- Full fine-tuning: 86 M parameters,
- LoRA ($r = 4$): 221 k (0.26%).

LoRA: <https://arxiv.org/pdf/2106.09685>

Focus on LoRA (PEFT)

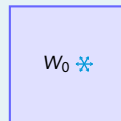
Low-Rank Adaptation

- Freeze the pre-trained matrix W_0
- Learn a **small correction**, written as $A \times B$ ($A \in \mathbb{R}^{h \times r}$ and $B \in \mathbb{R}^{r \times h}$, with r chosen by you ($r = 4, 8, 16, \dots$)).

Pre-trained weights

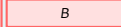
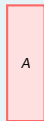
LoRA weights

$$x^{(l+1)} = W_0 x^{(l)} + AB x^{(l)}$$



$h \times h$ (large)

+



$h \times r$ and $r \times h$
(thin: $r \ll h$)

Example: a ViT

- Full fine-tuning: **86 M** parameters,
- LoRA ($r = 4$): **221 k** (0.26%).

LoRA: <https://arxiv.org/pdf/2106.09685>

What to choose?

Three questions, in this order

- 1 How much **labelled data** do you have?
- 2 How big is the **domain gap**?
- 3 What **compute** can you afford?

Two additional adaptations for LLMs

- **RAG** (Retrieval-Augmented Generation): adding documents in the context of the model/prompt. **Adds knowledge** without any training.
- **RLHF** / preference tuning: learn from human *comparisons* rather than labels. **Changes behaviour**, very important for AI assistants.

What to choose?

Three questions, in this order

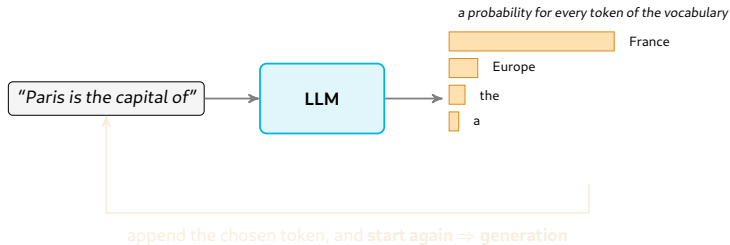
- 1 How much **labelled data** do you have?
- 2 How big is the **domain gap**?
- 3 What **compute** can you afford?

Two additional adaptations for LLMs

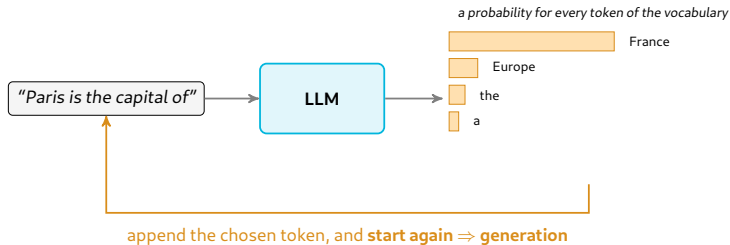
- **RAG** (Retrieval-Augmented Generation): adding documents in the context of the model/prompt. **Adds knowledge** without any training.
- **RLHF** / preference tuning: learn from human *comparisons* rather than labels. **Changes behaviour**, very important for AI assistants.

- 1 What is a Foundation Model?
- 2 Transformers
- 3 Self-supervised Learning
- 4 Multimodal Foundation Models
- 5 Adapting a Foundation Model
- 6 (Bonus) From LLMs to Assistants and Agents**
- 7 Conclusion

Using a Large Language Model (LLM) to generate text



Using a Large Language Model (LLM) to generate text



From text completion to instruction following

The problem

Pre-training teaches the model to **continue text**, not to **answer**.

Before (raw pre-trained)

You: What is the capital of France?

Model: What is the capital of Germany? What is the capital of Spain? ...

it just keeps the pattern going

After (instruction-tuned)

You: What is the capital of France?

Model: Paris.

it answers the actual question



From text completion to instruction following

The problem

Pre-training teaches the model to **continue text**, not to **answer**.

Before (raw pre-trained)

You: What is the capital of France?

Model: What is the capital of Germany? What is the capital of Spain? ...

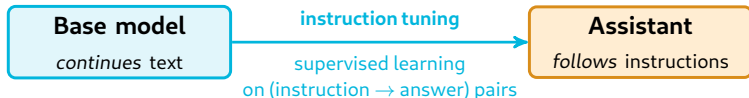
it just keeps the pattern going

After (instruction-tuned)

You: What is the capital of France?

Model: Paris.

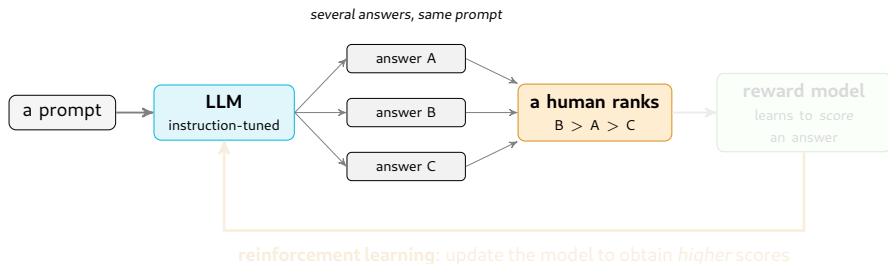
it answers the actual question



Reinforcement Learning from Human Feedback

RLHF: Put human in the loop

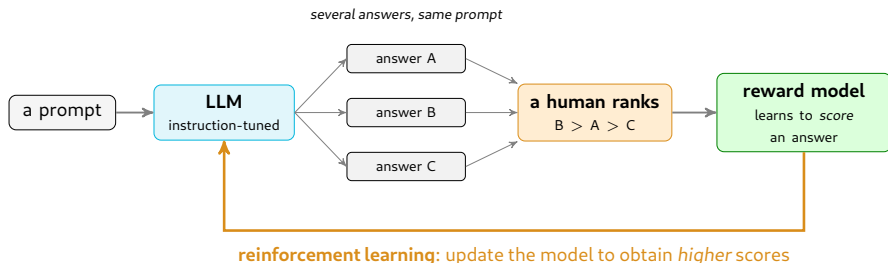
Using human evaluation to reward the model for producing better answers, aligns with pre-defined criteria (values, ethics, safety, ...).



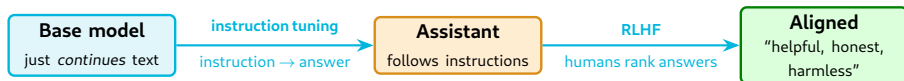
Reinforcement Learning from Human Feedback

RLHF: Put human in the loop

Using human evaluation to reward the model for producing better answers, aligns with pre-defined criteria (values, ethics, safety, ...).



Assistants are not just LLMs



RLHF and sycophancy

The model optimizes **what humans approve of**, based on pre-defined criteria.

Hence the behavior of some assistants: sympathetic, enthusiastic, ... but sometimes wrong to please you ("sycophancy").

RLHF is subject to the will of people training it: they may also be used to make a model antipathic, aggressive, manipulative, or else!

Reasoning models: Chain-of-Thought

Giving time to think

Taking time to write a reasoning process before answering.
Leads to better answers on arithmetic, logic, and planning problems,
but increases the computation cost.

Direct answer

Q: I have 5 balls, and I buy 6 cans of 7 balls. How many balls?

A: 43 ✗

it guesses in one shot

Chain-of-thought

Q: I have 5 balls, and I buy 6 cans of 7 balls. How many balls?

A: 6 cans $\times$ 7 = 42;
5 + 42 = 47. ✓

the steps first, then the answer

Chain-of-Thought largely participates in the recent progresses of LLMs!

Reasoning models: Chain-of-Thought

Giving time to think

Taking time to write a reasoning process before answering.
Leads to better answers on arithmetic, logic, and planning problems,
but increases the computation cost.

Direct answer

Q: I have 5 balls, and I buy 6 cans of 7 balls. How many balls?

A: 43 ✗

it guesses in one shot

Chain-of-thought

Q: I have 5 balls, and I buy 6 cans of 7 balls. How many balls?

A: 6 cans $\times$ 7 = 42;
5 + 42 = 47. ✓

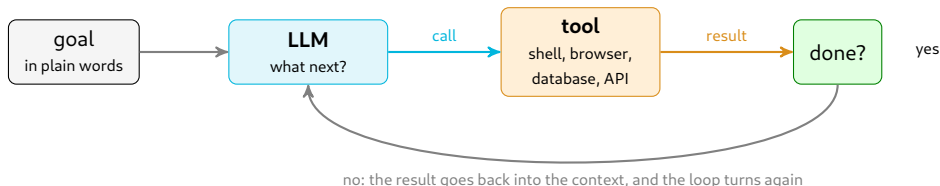
the steps first, then the answer

Chain-of-Thought largely participates in the recent progresses of LLMs!

Agentic AI: from answers to actions

Agents

An **agent** is an LLM that can call tools, observe the results, and decide what to do next, in a loop, until the goal is reached.

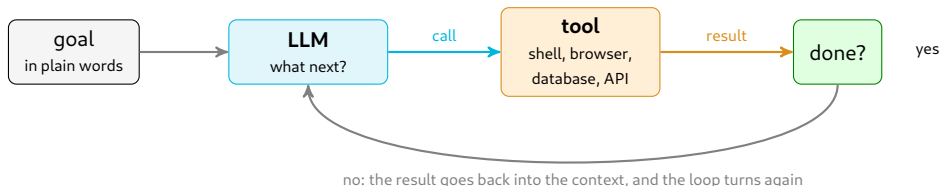


Nothing was added to the model. It was given a loop, and permission.

Agentic AI: from answers to actions

Agents

An **agent** is an LLM that can call tools, observe the results, and decide what to do next, in a loop, until the goal is reached.



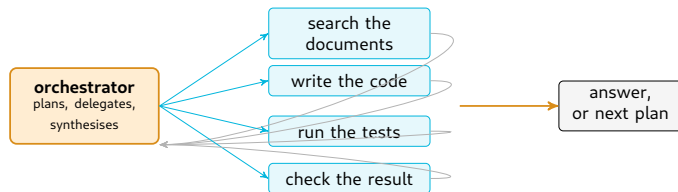
Nothing was added to the model. It was given a loop, and permission.

The LLM as orchestrator

The LLM orchestrates subtasks, but does not take action itself.

- 1 The LLM understands the overall task and breaks it down into smaller, manageable subtasks.
- 2 It **delegates** the pieces (to other model instances, or to deterministic tools).
- 3 It **recombines** the results, and decides whether to continue or to return the final answer.

Notice that the subtasks are decided at run time, not written in advance.

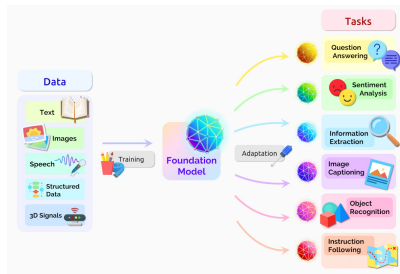


- 1 What is a Foundation Model?
- 2 Transformers
- 3 Self-supervised Learning
- 4 Multimodal Foundation Models
- 5 Adapting a Foundation Model
- 6 (Bonus) From LLMs to Assistants and Agents
- 7 Conclusion

What to remember

Foundation models

- Pre-trained **once**, on internet-scale data, and **reused**,
- Two ingredients: **Transformers** (attention) and **self-supervision** (pretext tasks),



Source: <https://blogs.nvidia.com/blog/what-are-foundation-models/>

Generalization is (mostly) solved now. Your job is **particularization** (e.g., adapting models!)

Lab Session 5

Part 1: CIFAR-10 through a **frozen** Vision Transformer

- Load a ViT from **HuggingFace**,
- Extract **embeddings**,
- **Probe** them with supervised learning models (lab 2), and compare with the CNN (lab 4).

Part 2: Your own project data

The same pipeline, with a backbone chosen for **your** data/modality.

Part 3: Adapting a model

Assess which adaptation techniques are most appropriate, and discover runnable code snippets.

Link to the lab: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab5/>

Adaptation tutorial: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/adapt/>